

SYSTEM DESIGN DOCUMENT

Task 2: Data Pipeline Design

1. Executive Summary

This document proposes a shipment data pipeline. The system ingests shipment records containing client, route, cargo, weight, value and status information, validates and normalizes the data, stores it in a reliable operational database and processes it into three business outputs:

1. Daily summary dashboard: operational visibility into shipment volumes, status, cargo, weight, value, and revenue.
2. Overdue shipment alerts: automatic detection and notification of shipments that require operational attention.
3. 30-day revenue forecast: a daily forecast of expected revenue for the next 30 days using historical revenue and known future shipments.

The design intentionally uses a relatively simple architecture which is Django, Celery, Redis, PostgreSQL, Python forecasting tools and Streamlit, rather than introducing heavyweight data-engineering infrastructure that is not necessary for the expected scale.

2. Business Requirements

2.1 Input Data

The pipeline must ingest shipment data containing at least:

Field	Description
shipment_id	Unique identifier for the shipment
client_name	Customer/client associated with the shipment
origin	Shipment origin
destination	Shipment destination
cargo_type	Type/category of cargo
weight_kg	Shipment weight
value_usd	Declared shipment/cargo value
status	Current shipment status

Additional operational fields such as ETA, vessel berth date, timestamps, and invoiced revenue are recommended because they support overdue detection, operational reporting, and revenue forecasting.

2.2 Required Outputs

The pipeline must produce:

- A daily summary dashboard.
- Overdue shipment alerts.
- A 30-day revenue forecast.

3. High-Level System Architecture

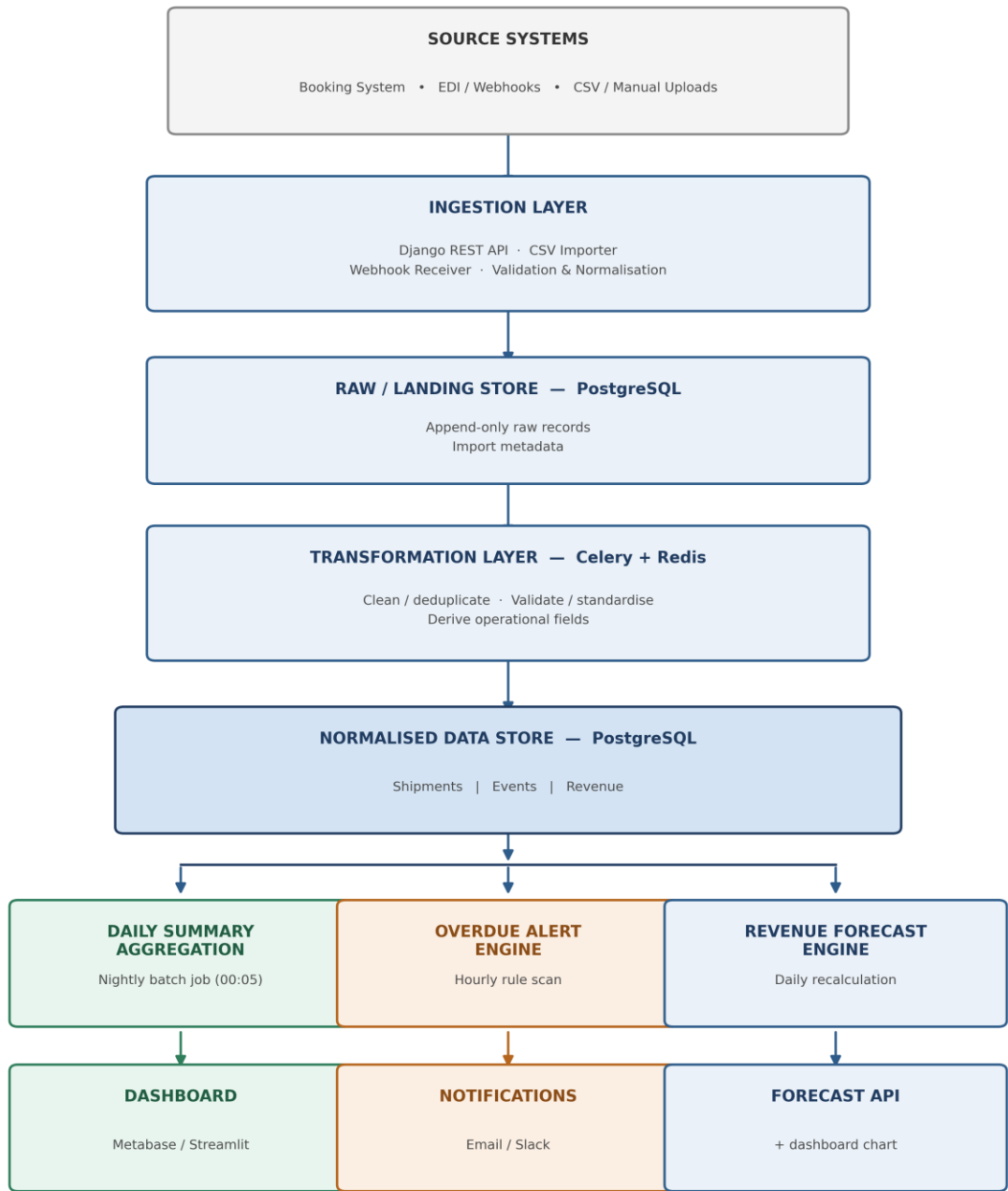


Figure 1: High-level shipment data pipeline architecture

3.1 Architecture Flow

The system follows a layered pipeline:

Ingest → Validate → Store Raw Data → Transform → Store Normalised Data → Analyse → Deliver Outputs

This separation ensures that downstream processes use one consistent source of truth. Raw records are retained for auditability, while cleaned and normalized records are used for analytics.

4. Data Model

4.1 shipments: Core Shipment Entity

Field	Type	Description
shipment_id	UUID / VARCHAR PK	Unique shipment identifier
client_id	FK	Reference to client
origin	VARCHAR	Origin port/city
destination	VARCHAR	Destination port/city
cargo_type	VARCHAR/ENUM	General, hazardous, perishable, vehicle, etc.
weight_kg	DECIMAL	Shipment weight
value_usd	DECIMAL	Declared cargo value
status	ENUM	Current shipment status
eta	TIMESTAMP	Expected delivery/arrival date
vessel_berth_date	TIMESTAMP	Expected vessel berth date where applicable
created_at	TIMESTAMP	Record creation time
updated_at	TIMESTAMP	Last update time

Example statuses:

BOOKED, DOCUMENTS_PENDING, DOCUMENTS_LODGED, IN_TRANSIT, ARRIVED_AT_PORT, CUSTOMS_CLEARANCE, OUT_FOR_DELIVERY, DELIVERED, DELAYED, CANCELLED.

4.2 shipment_events: Shipment History

An append-only event table records every important status transition.

Field	Type	Description
event_id	UUID PK	Event identifier
shipment_id	FK	Related shipment
previous_status	VARCHAR	Previous status
new_status	VARCHAR	New status
event_time	TIMESTAMP	Time of transition
source	VARCHAR	API, CSV, webhook, manual, etc.

Keeping history separately from the current shipment record allows the system to calculate transit times, detect periods without progress, and maintain an audit trail.

4.3 revenue_ledger: Revenue Data

Declared cargo value is not necessarily the same as revenue earned by the logistics company. Therefore, revenue is stored separately.

Field	Type	Description
revenue_id	UUID PK	Revenue record identifier
shipment_id	FK	Related shipment
invoice_date	DATE	Date revenue was invoiced
amount_usd	DECIMAL	Revenue amount
status	VARCHAR	Pending, invoiced, paid, etc.

4.4 Derived Output Tables

The analytics layer produces:

- daily_summary
- alerts
- revenue_forecasts

These are derived datasets and do not overwrite the operational shipment source of truth.

5. Ingestion and Data Validation

5.1 Supported Sources

The system can receive shipment data from:

- Booking/operations applications.
- REST API submissions.
- EDI or webhook feeds.
- Scheduled CSV uploads.
- Controlled manual imports.

5.2 Validation Rules

Incoming records should be validated before entering the normalized dataset.

Examples:

- shipment_id must be present and unique.
- client_name must not be empty.
- origin and destination must be valid non-empty values.
- weight_kg must be numeric and greater than zero.
- value_usd must be numeric and non-negative.
- status must belong to the supported status set.
- Dates must use a consistent timezone/format.
- Required fields must not contain placeholder values such as N/A where a real value is required.

Invalid records should be rejected or quarantined with an error reason rather than silently inserted.

5.3 Deduplication

Ingestion should be idempotent. Re-processing the same batch should not create duplicate shipments.

A practical initial strategy is:

Unique shipment identifier

+

source update timestamp

+

source/import identifier

The raw record is retained, while the normalized shipment is updated only when the incoming record represents a newer valid state.

6. Transformation and Processing Layer

Celery workers execute scheduled and asynchronous processing jobs using Redis as the message broker.

The transformation process performs:

4. Schema validation.
5. Data type conversion.
6. Text/category normalization.
7. Status normalization.
8. Duplicate detection.
9. Date/time normalization.
10. Calculation of derived fields.
11. Writing of normalized records.
12. Recording of processing errors.

A scheduled transformation job can run every 15 minutes for operational data, while daily aggregation and forecasting jobs run on their own schedules.

7. Daily Summary Dashboard

7.1 Purpose

The dashboard provides operations staff and management with a daily view of shipment activity and business performance.

7.2 Metrics

The daily summary should include:

Shipment Operations

- Total active shipments.
- Total shipments created today.
- Shipments by status.
- Shipments by cargo type.
- Shipments by origin and destination.
- Delivered shipments.
- Delayed shipments.
- Shipments currently on customs hold.

Volume and Value

- Total weight handled.
- Total declared cargo value.
- Average shipment weight.
- Average shipment value.

Transit Performance

- Average transit time.

- Average time in current status.
- Number of shipments exceeding ETA.
- Number of shipments approaching ETA.

Revenue

- Revenue invoiced today.
- Revenue invoiced during the current month.
- Revenue by client.
- Revenue by shipment/cargo category where applicable.

7.3 Dashboard Generation

A scheduled Celery job runs shortly after midnight to materialize the previous day's summary into `daily_summary`.

The dashboard can be implemented using:

- Metabase for the fastest operational deployment.
- Streamlit if a custom/branded analytical interface is required.

The dashboard reads from normalized/derived tables and does not directly modify shipment records.

8. Overdue Shipment Alert System

8.1 Purpose

The alert engine identifies shipments that require operational intervention and sends notifications to the responsible team.

8.2 Alert Rules

A shipment is considered at risk or overdue when one or more rules are triggered.

Rule 1: ETA Exceeded

`status = IN_TRANSIT`
`AND current_time > eta`

Severity: HIGH

Rule 2: Customs Hold

`status = CUSTOMS_CLEARANCE / CUSTOMS_HOLD`
`AND no status progress for > 48 hours`

Severity: HIGH

Rule 3: Documents Not Lodged

`required documents not lodged`
`AND vessel_berth_date - current_time <= 7 days`

Severity: HIGH

Rule 4: Delayed Status

status = DELAYED

Severity: HIGH

Additional rules can be introduced as operational data becomes available.

8.3 Alert Processing

An hourly Celery task:

13. Reads current shipments and shipment events.
14. Evaluates alert rules.
15. Creates an alert when a rule is triggered.
16. Avoids creating duplicate open alerts for the same shipment/rule.
17. Sends the notification.
18. Marks the alert resolved when the underlying condition is cleared.

Example alert:

ALERT: Shipment CLF-2026-002 is overdue.

Status: IN_TRANSIT

ETA: 04 Sep 2026

Current time: 06 Sep 2026

Severity: HIGH

Action: Contact carrier/operations team.

8.4 Notification Channels

Initial implementation:

- Email via SMTP.
- Slack webhook.

Future options:

- Microsoft Teams.
- SMS.
- In-app notifications.

9. 30-Day Revenue Forecasting

9.1 Objective

The forecasting component estimates expected logistics revenue for each of the next 30 days.

The forecast should support planning rather than be treated as a guaranteed financial figure.

9.2 Input Data

The forecasting model uses:

- Historical invoiced revenue.
- Revenue by day.
- Day-of-week patterns.
- Recent revenue trends.
- Known future/booked shipments.
- Expected shipment ETA/berth date.
- Optional client or cargo-level revenue history when sufficient data exists.

9.3 Forecasting Approach

A staged approach is recommended.

Baseline

First aggregate historical revenue into a daily time series and establish a weighted moving-average baseline with day-of-week seasonality.

This provides a simple, explainable model and establishes a benchmark against which more advanced models can be evaluated.

Confirmed Pipeline Adjustment

Known future shipments with expected revenue are incorporated into the forecast.

This is important because a shipment already booked for a future period represents information that a purely historical time-series model cannot see.

Conceptually:

Final Forecast

=

Historical/Statistical Forecast

+

Expected Revenue from Confirmed Pipeline

Care must be taken to avoid double-counting revenue already represented in the historical series.

Upgrade Path

Once sufficient historical data is available, models such as Prophet or statsmodels-based forecasting can be evaluated against the baseline.

Model selection should be based on backtesting performance rather than assuming a more complex model is automatically better.

9.4 Forecast Output

The system produces:

Date	Forecast Revenue	Lower Bound	Upper Bound
Day 1	Forecast	Interval	Interval
Day 2	Forecast	Interval	Interval
...	...	...	...
Day 30	Forecast	Interval	Interval

The forecast is recalculated daily.

9.5 Forecast Evaluation

The forecasting system should be evaluated using historical backtesting.

Recommended metrics:

- MAE: Mean Absolute Error.
- RMSE: Root Mean Squared Error.
- MAPE: Mean Absolute Percentage Error, where appropriate.

A simple baseline should be retained so that management can see whether the forecasting model provides meaningful improvement.

10. Technology Stack

Layer	Technology	Purpose
API/Ingestion	Django REST Framework	Shipment data ingestion
Background processing	Celery	Asynchronous/scheduled jobs
Message broker	Redis	Celery task queue
Database	PostgreSQL	Raw, normalized, event and revenue data
Data processing	Python + pandas	Transformation and aggregation
Forecasting	statsmodels / Prophet	Revenue forecasting
Dashboard	Metabase / Streamlit	Operational reporting
Notifications	SMTP / Slack webhook	Overdue alerts
Testing	pytest / Django tests	Pipeline and rule validation
Deployment	Docker / cloud VM or platform	Reproducible deployment

Heavy technologies such as Kafka or Airflow are intentionally excluded from the initial implementation. They can be introduced later if data volume, workflow complexity, or operational requirements justify them.

11. Scheduling

A simple scheduling model is sufficient for the proposed system.

Job	Frequency	Purpose
Shipment ingestion	Real-time / scheduled	Receive new shipment data
Transformation	Every 15 minutes	Clean and normalize incoming records

Overdue scan	Hourly	Detect operational risks
Daily aggregation	Daily at 00:05	Generate daily dashboard metrics
Revenue forecast	Daily after aggregation	Produce next 30-day forecast
Alert cleanup	Hourly/daily	Resolve or archive completed alerts

12. Error Handling and Recovery

The pipeline should fail safely.

Ingestion failures

Invalid records are placed into an error/quarantine area with:

- Record identifier.
- Import identifier.
- Error message.
- Timestamp.

Processing failures

Celery tasks should support retries for transient failures.

Database failures

Transactions should prevent partially written normalized records.

Forecast failures

If the forecasting model fails, the system should retain the most recent successful forecast and expose an error status rather than displaying fabricated values.

Alert failures

If notification delivery fails, the alert remains stored in the alert queue so it can be retried.

13. Data Quality and Governance

The following controls are recommended:

- Preserve raw input records.
- Maintain shipment event history.
- Validate required fields.
- Use controlled status values.
- Track record creation and update timestamps.
- Make ingestion idempotent.
- Log transformation failures.
- Separate operational data from derived analytics.
- Restrict dashboard access according to user roles.
- Protect client and commercial information.
- Back up PostgreSQL data regularly.

This is particularly important because shipment, client, and financial data should be traceable and reproducible.

14. Security Considerations

The system should include:

- Authenticated ingestion endpoints.
- Role-based dashboard access.
- HTTPS for API communication.
- Environment variables for credentials and secrets.
- Database access controls.
- Validation of uploaded CSV files.
- Rate limiting on externally exposed endpoints where appropriate.
- Audit logging for important data changes.

Sensitive client and financial information should not be exposed through public dashboards or logs.

15. Scalability and Future Improvements

The initial design is intended for a small-to-medium logistics operation and can scale incrementally.

Phase 1

- PostgreSQL.
- Django REST Framework.
- Celery + Redis.
- Metabase/Streamlit.
- Basic forecasting.
- Email/Slack alerts.

Phase 2

- Automated EDI integrations.
- More detailed operational KPIs.
- Improved forecast models.
- Role-specific dashboards.
- Automated data-quality monitoring.

Phase 3

If data volume and event throughput become substantially larger:

- Kafka for high-volume event streaming.
- Airflow or another workflow orchestrator for complex data pipelines.
- Dedicated analytical warehouse/lakehouse.
- More advanced forecasting and anomaly detection.

The architecture therefore avoids premature complexity while leaving clear upgrade paths.

16. End-to-End Example

Consider a shipment with:

Shipment ID: CLF-2026-002

Client: Example Importer

Origin: Shanghai

Destination: Durban

Cargo: General Goods

Weight: 8,500 kg

Value: USD 45,000

Status: IN_TRANSIT

ETA: 04 September 2026

Step 1: Ingestion

The shipment is received through the REST API or CSV importer.

Step 2: Validation

The pipeline verifies the required fields, numeric values, status and dates.

Step 3: Storage

The raw record is retained and the valid shipment is written to the normalized shipments table.

Step 4: Event Processing

Status changes are recorded in shipment_events.

Step 5 Overdue Detection

If the current date is later than the ETA while the shipment remains IN_TRANSIT, the overdue rule triggers.

Step 6: Alert

A HIGH severity alert is created and sent to the operations team.

Step 7: Dashboard

The shipment contributes to the daily counts, weight/value totals, and overdue shipment KPI.

Step 8: Forecast

If the shipment is associated with known future revenue, its expected revenue can contribute to the confirmed pipeline component of the 30-day revenue forecast.

17. Key Design Decisions

17.1 One normalized source of truth

The dashboard, alert engine and forecasting process should consume the normalized data layer rather than independently processing raw files.

17.2 Raw data is preserved

Raw input is retained so that processing errors can be investigated and historical records can be reproduced.

17.3 Derived data is separated

Daily summaries, alerts and forecasts are derived outputs and should not overwrite operational shipment records.

17.4 Rules before complexity

Overdue detection is implemented using transparent business rules rather than an ML model. This makes alerts explainable and easy for operations staff to validate.

17.5 Forecasting starts with a baseline

A simple statistical baseline is established before introducing more complex forecasting models. This makes model improvement measurable.

18. Conclusion

The proposed pipeline provides a practical end-to-end solution for turning shipment data into operational intelligence.

The architecture:

Ingests → validates → stores → transforms → analyses → delivers

It directly satisfies the three requirements of Task 2:

- Daily summary dashboard for operational visibility.
- Overdue shipment alerts for proactive intervention.
- 30-day revenue forecasting for planning and decision-making.

The design favors reliability, auditability, explainability and maintainability over unnecessary infrastructure complexity.